SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning (BL 5)

Assessment Tool Weightage: 40%

QUESTIONS: 15 **Total Marks: 25**

Annexure A

Coding Challenge

Code of Conduct:

I V.Madhulika (192312620) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

V.Madhulika

Signature of the student

Q1. Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

Answer:

```
import numpy as np
from collections import Counter

# Small dataset: [feature1, feature2] -> class label
X_train = np.array([[1,2],[2,3],[3,3],[6,5],[7,7],[8,6],[2,1],[7,8]])
y_train = np.array(['A','A','A','B','B','B','A','B'])

X_test  = np.array([[3,2],[6,6],[1,1],[8,7]])
y_true  = np.array(['A','B','A','B'])   # ground truth for accuracy check

def euclidean(a, b):
    return np.sqrt(np.sum((a - b) ** 2))

def knn_predict(X_train, y_train, x_query, k):
    distances = [euclidean(x_query, x) for x in X_train]
    k_idx = np.argsort(distances)[:k]
    k_labels = [y_train[i] for i in k_idx]
    vote = Counter(k_labels).most_common(1)[0][0]
    return vote

k = int(input("Enter value of K: "))
predictions = [knn_predict(X_train, y_train, x, k) for x in X_test]

print("Predicted labels:", predictions)
accuracy = np.mean(np.array(predictions) == y_true) * 100
print(f"Accuracy: {accuracy:.2f}%")
```

The program computes the Euclidean distance from a query point to every training point, sorts them, picks the K closest neighbours, and assigns the majority class among them. Increasing K smooths the decision boundary and reduces sensitivity to noise, but a very large K can cause under-fitting by pulling in points from other classes; a very small K (e.g. K=1) fits the training data closely but is sensitive to outliers. Accuracy is reported by comparing predictions with the known ground-truth labels of the test set.

Q2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.

Answer:

```
import numpy as np

# Dataset
X = np.array([1,2,3,4,5,6,7,8,9,10], dtype=float)
y = np.array([1.5,2.0,3.2,4.1,5.3,6.0,7.4,8.1,9.0,10.5], dtype=float)
```

```
X_b = np.c_[np.ones_like(X), X]   # add bias column

def gaussian_kernel(x, x_query, tau):
    return np.exp(-((x - x_query) ** 2) / (2 * tau ** 2))

def locally_weighted_regression(X_b, y, x_query, tau):
    m = X_b.shape[0]
    W = np.diag([gaussian_kernel(X_b[i,1], x_query, tau) for i in range(m)])
    theta = np.linalg.pinv(X_b.T @ W @ X_b) @ X_b.T @ W @ y
    x_q_b = np.array([1, x_query])
    return x_q_b @ theta

# Standard linear regression (closed form)
theta_lin = np.linalg.pinv(X_b.T @ X_b) @ X_b.T @ y

query_points = [2.5, 5.5, 8.5]
tau = 1.0
print("Query | LWR Prediction | Linear Regression Prediction")
for q in query_points:
    lwr_pred = locally_weighted_regression(X_b, y, q, tau)
    lin_pred = np.array([1, q]) @ theta_lin
    print(f"{q:5} | {lwr_pred:14.3f} | {lin_pred:28.3f}")
```

Locally weighted regression fits a fresh weighted linear model for every query point, using a Gaussian kernel so nearby training points get near-weight 1 and distant points get near-weight 0. This lets the model follow local curvature that a single global linear regression line cannot capture. In the comparison, standard linear regression gives the same prediction slope everywhere, while LWR adapts locally — closely tracking the data near each query point, at the cost of higher computation since a new regression is solved for every prediction. The bandwidth τ controls the trade-off: small τ gives a very local, wiggly fit; large τ approaches ordinary linear regression.

Q3. Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.

Answer:

```
import numpy as np

X = np.array([0,1,2,3,4,5,6,7,8,9], dtype=float).reshape(-1,1)
y = np.array([0,0.8,0.9,0.1,-0.8,-1,-0.9,-0.1,0.8,1], dtype=float)  # ~ sin-like pattern

def rbf(x, c, sigma):
    return np.exp(-np.linalg.norm(x - c) ** 2 / (2 * sigma ** 2))

class RBFNetwork:
    def __init__(self, n_centers=4, sigma=1.5):
        self.n_centers = n_centers
        self.sigma = sigma
```

```
    def fit(self, X, y):
        idx = np.linspace(0, len(X) - 1, self.n_centers).astype(int)
        self.centers = X[idx]
        G = np.array([[rbf(x, c, self.sigma) for c in self.centers] for x in X])
        self.weights = np.linalg.pinv(G) @ y

    def predict(self, X):
        G = np.array([[rbf(x, c, self.sigma) for c in self.centers] for x in X])
        return G @ self.weights

for sigma in [0.5, 1.5, 3.0]:
    for n_centers in [2, 4, 6]:
        model = RBFNetwork(n_centers=n_centers, sigma=sigma)
        model.fit(X, y)
        preds = model.predict(X)
        mse = np.mean((preds - y) ** 2)
        print(f"centers={n_centers}, sigma={sigma} -> MSE={mse:.4f}")
```

Each hidden unit computes a Gaussian activation centred at a fixed prototype; the output layer is a linear combination of these activations, so training reduces to a least-squares fit of the output weights. Increasing the number of centres lets the network represent finer detail in the data (lower training error) but risks over-fitting and higher computational cost. The width (sigma) controls how far each Gaussian's influence reaches: a small sigma creates narrow, highly local bumps that can produce a jagged, over-fit function with gaps between centres, while a large sigma over-smooths the output and can blur distinct patterns together. The experiment above sweeps both parameters and reports MSE so the best combination for the dataset can be identified.

Q4. Create a Python program that implements a basic Case-Based Learning system. Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.

Answer:

```
import numpy as np

# Case base: (problem features, known solution)
case_base = [
    ({'symptom_fever': 1, 'symptom_cough': 1, 'symptom_fatigue': 0}, 'Flu'),
    ({'symptom_fever': 0, 'symptom_cough': 1, 'symptom_fatigue': 0}, 'Common Cold'),
    ({'symptom_fever': 1, 'symptom_cough': 0, 'symptom_fatigue': 1}, 'Malaria'),
    ({'symptom_fever': 0, 'symptom_cough': 0, 'symptom_fatigue': 1}, 'Fatigue Disorder'),
]

def to_vector(case):
    return np.array(list(case.values()))

def euclidean_similarity(query, case):
    dist = np.linalg.norm(to_vector(query) - to_vector(case))
    return 1 / (1 + dist)   # convert distance to a similarity score in (0,1]
```

```
def retrieve_best_case(query, case_base):
    scored = [(euclidean_similarity(query, c), sol) for c, sol in case_base]
    scored.sort(key=lambda x: x[0], reverse=True)
    return scored[0]

new_query = {'symptom_fever': 1, 'symptom_cough': 1, 'symptom_fatigue': 0}
similarity, solution = retrieve_best_case(new_query, case_base)
print(f"Most similar case similarity score: {similarity:.3f}")
print(f"Retrieved solution: {solution}")
```

Case-based learning stores previously solved problems ("cases") together with their solutions and, for a new problem, retrieves the case whose feature vector is most similar and reuses its solution. Similarity here is computed by taking the Euclidean distance between the query's feature vector and each stored case's feature vector, then converting distance into a bounded similarity score (1/(1+distance)) so that closer cases score nearer 1. The case with the highest similarity is retrieved and its stored solution is returned as the answer, mirroring how a human expert would recall the most similar past experience to solve a new problem.

Q5. Write a Python program to implement both KNN and RBF models on the same dataset. Compare their predictions for given test inputs. Analyze differences in computation, accuracy, and output behavior, and display results clearly for better comparison.

Answer:

```
import numpy as np
from collections import Counter
import time

X_train = np.array([[1,2],[2,1],[3,3],[6,5],[7,7],[8,6],[2,2],[7,8]])
y_train = np.array([0,0,0,1,1,1,0,1])
X_test  = np.array([[2,3],[7,6]])
y_true  = np.array([0,1])

# ---- KNN ----
def knn_predict(x, k=3):
    d = np.linalg.norm(X_train - x, axis=1)
    idx = np.argsort(d)[:k]
    return Counter(y_train[idx]).most_common(1)[0][0]

# ---- RBF classifier ----
def rbf(x, c, sigma=2.0):
    return np.exp(-np.linalg.norm(x - c) ** 2 / (2 * sigma ** 2))

centers = X_train
G_train = np.array([[rbf(x, c) for c in centers] for x in X_train])
weights = np.linalg.pinv(G_train) @ y_train

def rbf_predict(x):
    g = np.array([rbf(x, c) for c in centers])
```

```
    return 1 if (g @ weights) >= 0.5 else 0

t0 = time.time()
knn_preds = [knn_predict(x) for x in X_test]
t_knn = time.time() - t0

t0 = time.time()
rbf_preds = [rbf_predict(x) for x in X_test]
t_rbf = time.time() - t0

print("Test Input | KNN Pred | RBF Pred | Actual")
for i, x in enumerate(X_test):
    print(f"{x} | {knn_preds[i]} | {rbf_preds[i]} | {y_true[i]}")

print(f"KNN accuracy: {np.mean(np.array(knn_preds)==y_true)*100:.1f}%  time={t_knn:.6f}s")
print(f"RBF accuracy: {np.mean(np.array(rbf_preds)==y_true)*100:.1f}%  time={t_rbf:.6f}s")
```

KNN is a lazy, instance-based learner: it stores all training data and defers computation to prediction time, comparing the query against every stored point, so its per-query cost grows with dataset size and it has no separate training phase. RBF is an eager learner: it fixes a set of Gaussian centres and solves for output weights once during training, so prediction only involves evaluating the (usually smaller) set of basis functions, making it faster at inference time on large datasets. On smooth, well-separated data both give similar accuracy, but KNN's decision boundary is more jagged/local while RBF's boundary is smoother because it is built from continuous Gaussian activations combined linearly. KNN is very sensitive to the choice of K and to noisy neighbours, whereas RBF's behaviour is governed by the number and width of its centres.

Q6. Write a Python program to implement the Decision Tree algorithm for classification. Use a small dataset and construct the tree using information gain or Gini index. Allow prediction for test instances and display accuracy. Visualize the tree structure and analyze how feature selection impacts classification performance.

Answer:

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Small dataset: [Outlook(0=Sunny,1=Overcast,2=Rain), Temp(0=Hot,1=Mild,2=Cool),
Windy(0=No,1=Yes)]
X = np.array([
    [0,0,0],[0,0,1],[1,0,0],[2,1,0],[2,2,0],
    [2,2,1],[1,2,1],[0,1,0],[0,2,0],[2,1,1],
    [0,1,1],[1,1,0],[1,0,1],[2,1,1]
])
y = np.array([0,0,1,1,1,0,1,0,1,0,1,1,1,0])  # 0=No Play, 1=Play

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
clf = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

print("Predicted:", y_pred)
print("Actual   :", y_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred)*100:.2f}%")
print("Feature importances (Outlook, Temp, Windy):", clf.feature_importances_)

plt.figure(figsize=(10,6))
plot_tree(clf, feature_names=['Outlook','Temp','Windy'], class_names=['No','Yes'], filled=True)
plt.savefig('decision_tree.png')
```

The tree is built greedily: at each node the Gini index (or information gain, based on entropy) is computed for every candidate feature/threshold split, and the split that most reduces impurity is chosen, recursively partitioning the data until nodes are pure or a stopping criterion (max_depth) is reached. The `feature_importances_` attribute shows how much each feature contributed to reducing impurity across the whole tree; features that are chosen near the root (e.g. Outlook) tend to dominate classification because they separate the largest portion of the data first. Poor feature selection (irrelevant or highly correlated features) increases tree depth and the risk of over-fitting, while informative features let a shallow tree reach high accuracy with better generalisation.

Q7. Develop a Python program to implement a Naïve Bayes classifier for a given dataset. Compute prior and conditional probabilities, and classify a test instance. Display predicted class labels and evaluate performance using accuracy. Compare results with and without Laplace smoothing and analyze its impact on classification.

Answer:

```
import numpy as np
from collections import defaultdict

# Dataset: categorical features -> class
data = [
   (('Sunny','Hot','High'), 'No'), (('Sunny','Hot','High'), 'No'),
   (('Overcast','Hot','High'), 'Yes'), (('Rain','Mild','High'), 'Yes'),
   (('Rain','Cool','Normal'), 'Yes'), (('Rain','Cool','Normal'), 'No'),
   (('Overcast','Cool','Normal'), 'Yes'), (('Sunny','Mild','High'), 'No'),
   (('Sunny','Cool','Normal'), 'Yes'), (('Rain','Mild','Normal'), 'Yes'),
]

classes = set(c for _, c in data)
feature_vals = [set(f[i] for f, _ in data) for i in range(3)]

def train_nb(data, laplace=True):
   priors, cond = {}, defaultdict(lambda: defaultdict(dict))
   for c in classes:
      class_data = [f for f, lbl in data if lbl == c]
      priors[c] = len(class_data) / len(data)
```

```
        for i in range(3):
            vals = feature_vals[i]
            for v in vals:
                count = sum(1 for f in class_data if f[i] == v)
                if laplace:
                    cond[c][i][v] = (count + 1) / (len(class_data) + len(vals))
                else:
                    cond[c][i][v] = count / len(class_data) if len(class_data) else 0
    return priors, cond

def predict(instance, priors, cond):
    scores = {}
    for c in classes:
        p = priors[c]
        for i, v in enumerate(instance):
            p *= cond[c][i].get(v, 1e-6)
        scores[c] = p
    return max(scores, key=scores.get)

test = ('Sunny','Cool','High')
for smoothing in [False, True]:
    priors, cond = train_nb(data, laplace=smoothing)
    pred = predict(test, priors, cond)
    print(f"Laplace smoothing={smoothing} -> Predicted: {pred}")

correct = 0
priors, cond = train_nb(data, laplace=True)
for f, lbl in data:
    if predict(f, priors, cond) == lbl:
        correct += 1
print(f"Training accuracy (with smoothing): {correct/len(data)*100:.2f}%")
```

Naive Bayes assumes conditional independence between features given the class, so it multiplies the prior probability of each class by the conditional probability of every observed feature value under that class, and picks the class with the highest resulting posterior score. Without Laplace smoothing, any feature value that never appeared with a class in training gets a conditional probability of exactly zero, which forces the whole product to zero regardless of how well the other features match — the model can wrongly rule out a class purely due to sparse data. Laplace (add-one) smoothing adds a small pseudo-count to every combination, so unseen feature/class pairs get a small non-zero probability instead of zero; this makes the classifier more robust on small or sparse datasets at the cost of slightly biasing well-observed probabilities toward uniformity.

Q8. Write a Python program to implement Logistic Regression for binary classification. Train the model using gradient descent and predict outputs for test data. Display the sigmoid function output and classification results. Evaluate the model using accuracy and discuss how learning rate affects convergence and performance.

Answer:

```
import numpy as np
```

```
np.random.seed(0)
X = np.array([[1,1],[2,1],[2,3],[3,2],[6,5],[7,7],[8,6],[7,5]], dtype=float)
y = np.array([0,0,0,0,1,1,1,1], dtype=float)
X_b = np.c_[np.ones(len(X)), X]

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def train_logistic(X, y, lr=0.1, epochs=2000):
    theta = np.zeros(X.shape[1])
    for _ in range(epochs):
        z = X @ theta
        preds = sigmoid(z)
        grad = X.T @ (preds - y) / len(y)
        theta -= lr * grad
    return theta

for lr in [0.001, 0.1, 1.0]:
    theta = train_logistic(X_b, y, lr=lr, epochs=2000)
    probs = sigmoid(X_b @ theta)
    preds = (probs >= 0.5).astype(int)
    acc = np.mean(preds == y) * 100
    print(f"lr={lr}: sigmoid outputs={np.round(probs,2)} accuracy={acc:.1f}%")
```

Logistic regression models the probability of the positive class as the sigmoid of a linear combination of features, and gradient descent updates the weights in the direction that reduces the log-loss between predicted probabilities and true labels. A very small learning rate (0.001) converges slowly and may not reach the optimum within the given epochs, giving underfit predictions close to 0.5. A moderate learning rate (0.1) converges smoothly to a good decision boundary. A learning rate that is too large (1.0) can overshoot the minimum and cause the loss to oscillate or diverge, degrading accuracy. Choosing an appropriate learning rate (or using an adaptive schedule) is therefore critical for both fast and stable convergence.

Q9. Create a Python program to implement the K-Means clustering algorithm. Initialize cluster centers, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyze how the number of clusters and initialization methods affect clustering results.

Answer:

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(1)
X = np.vstack([
    np.random.randn(20, 2) + [2, 2],
    np.random.randn(20, 2) + [8, 8],
    np.random.randn(20, 2) + [2, 8],
])
```

```
def kmeans(X, k, n_iter=100, seed=0):
    rng = np.random.RandomState(seed)
    centers = X[rng.choice(len(X), k, replace=False)]
    for _ in range(n_iter):
        distances = np.linalg.norm(X[:, None] - centers[None, :], axis=2)
        labels = np.argmin(distances, axis=1)
        new_centers = np.array([X[labels == i].mean(axis=0) if np.any(labels==i) else centers[i] for i in
range(k)])
        if np.allclose(new_centers, centers):
            break
        centers = new_centers
    return labels, centers

for k in [2, 3, 4]:
    labels, centers = kmeans(X, k, seed=42)
    plt.figure()
    plt.scatter(X[:,0], X[:,1], c=labels, cmap='viridis')
    plt.scatter(centers[:,0], centers[:,1], c='red', marker='X', s=200)
    plt.title(f"K-Means with k={k}")
    plt.savefig(f"kmeans_k{k}.png")

    inertia = sum(np.linalg.norm(X[labels==i] - centers[i], axis=1).sum() for i in range(k))
    print(f"k={k}: inertia={inertia:.2f}")
```

K-Means alternates between assigning each point to its nearest centroid and recomputing centroids as the mean of the points assigned to them, until the assignment stabilises. Choosing k too small merges genuinely separate groups into one cluster (high inertia, poor separation), while choosing k too large splits natural groups unnecessarily and can produce empty or unstable clusters; the elbow method (plotting inertia against k) is a common way to pick a reasonable k. The initialisation of centroids also matters: random initialisation can converge to a poor local optimum depending on the seed, which is why running K-Means multiple times with different seeds (or using K-Means++ initialisation, which spreads initial centroids apart) generally produces more stable, better-separated clusters.

Q10. Write a Python program to implement a simple Genetic Algorithm to optimize a mathematical function. Initialize a population, perform selection, crossover, and mutation operations, and evolve solutions over generations. Display fitness values and best solution, and analyze how parameters influence convergence speed and solution quality.

Answer:

```
import numpy as np

def fitness(x):
    return -(x - 3) ** 2 + 9   # maximum at x = 3, value = 9

def genetic_algorithm(pop_size=20, generations=50, mutation_rate=0.1, bounds=(-10,10)):
    rng = np.random.RandomState(0)
    population = rng.uniform(bounds[0], bounds[1], pop_size)
```

```
    for gen in range(generations):
        fitness_vals = fitness(population)
        # Selection: tournament of 2
        parents = []
        for _ in range(pop_size):
            i, j = rng.randint(0, pop_size, 2)
            parents.append(population[i] if fitness_vals[i] > fitness_vals[j] else population[j])
        parents = np.array(parents)

        # Crossover (arithmetic)
        offspring = []
        for i in range(0, pop_size, 2):
            p1, p2 = parents[i], parents[(i+1) % pop_size]
            alpha = rng.rand()
            offspring.append(alpha*p1 + (1-alpha)*p2)
            offspring.append(alpha*p2 + (1-alpha)*p1)
        offspring = np.array(offspring)

        # Mutation
        mutation_mask = rng.rand(pop_size) < mutation_rate
        offspring[mutation_mask] += rng.normal(0, 1, mutation_mask.sum())
        offspring = np.clip(offspring, bounds[0], bounds[1])

        population = offspring
        if gen % 10 == 0:
            best = population[np.argmax(fitness(population))]
            print(f"Gen {gen}: best x={best:.3f}, fitness={fitness(best):.3f}")

    best = population[np.argmax(fitness(population))]
    return best, fitness(best)

best_x, best_fit = genetic_algorithm()
print(f"Best solution: x={best_x:.3f}, fitness={best_fit:.3f}")
```

The algorithm evolves a population of candidate solutions toward the maximum of a fitness function using tournament selection (favouring fitter individuals as parents), arithmetic crossover (blending two parents to produce offspring), and Gaussian mutation (randomly perturbing some offspring to maintain diversity). A larger population and more generations generally improve solution quality and reduce the chance of premature convergence to a suboptimal point, at the cost of more computation. A mutation rate that is too low can cause the population to converge prematurely and get stuck, missing the true optimum, while a mutation rate that is too high injects too much randomness and prevents the population from converging at all. Tuning population size, generations, and mutation rate together controls the trade-off between convergence speed and solution quality.

Q11. Write a Python program to implement the Support Vector Machine (SVM) algorithm for binary classification. Use a simple dataset and implement a linear kernel. Train the model, classify test instances, and display accuracy. Analyze how the margin and support vectors influence classification performance.

Answer:

```
import numpy as np
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

np.random.seed(0)
X = np.vstack([np.random.randn(25,2)+[2,2], np.random.randn(25,2)+[7,7]])
y = np.array([0]*25 + [1]*25)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

model = SVC(kernel='linear', C=1.0)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual   :", y_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred)*100:.2f}%")
print("Number of support vectors:", model.n_support_)
print("Margin width (2/||w||):", 2 / np.linalg.norm(model.coef_))
```

The linear-kernel SVM finds the hyperplane that maximises the margin between the two classes; only the points closest to the boundary — the support vectors — determine the position of that hyperplane, so the model is inherently robust to points far from the boundary. A wider margin generally indicates better generalisation to unseen data because the decision boundary sits further from both classes, reducing sensitivity to small perturbations. The regularisation parameter C controls the trade-off: a small C allows a wider margin at the cost of tolerating some misclassified training points (soft margin), while a large C forces a narrower margin that tries to classify every training point correctly, which can overfit noisy data. The number of support vectors reported also indicates model complexity — fewer support vectors typically mean a simpler, more generalisable boundary.

Q12. Develop a Python program to implement Principal Component Analysis (PCA) for dimensionality reduction. Apply PCA on a dataset, reduce it to two principal components, and visualize the transformed data. Compare model performance before and after dimensionality reduction and analyze variance retention.

Answer:

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

data = load_iris()
X, y = data.data, data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

# Model on original (4D) features
clf_full = LogisticRegression(max_iter=500).fit(X_train, y_train)
acc_full = accuracy_score(y_test, clf_full.predict(X_test))

# PCA to 2 components
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

clf_pca = LogisticRegression(max_iter=500).fit(X_train_pca, y_train)
acc_pca = accuracy_score(y_test, clf_pca.predict(X_test_pca))

print(f"Accuracy with all 4 features : {acc_full*100:.2f}%")
print(f"Accuracy with 2 PCA components: {acc_pca*100:.2f}%")
print("Explained variance ratio (PC1, PC2):", pca.explained_variance_ratio_)
print("Total variance retained:", sum(pca.explained_variance_ratio_)*100, "%")

plt.figure()
plt.scatter(X_train_pca[:,0], X_train_pca[:,1], c=y_train, cmap='viridis')
plt.xlabel("PC1"); plt.ylabel("PC2")
plt.savefig("pca_plot.png")
```

PCA finds the orthogonal directions (principal components) along which the data varies most, and projecting onto the top two components compresses the dataset from 4 dimensions down to 2 while retaining as much variance as possible. In this experiment the first two components typically retain around 95-98% of the total variance for the Iris dataset, so classification accuracy after reduction is close to, and sometimes matches, the accuracy achieved with all four original features — showing that most of the discriminative information is preserved. The trade-off is that a small amount of variance (and potentially some class-separating information) is discarded, so PCA is most useful when the goal is to reduce computation and visualise data while accepting a small, usually acceptable, drop in accuracy.

Q13. Write a Python program to implement the Hierarchical Clustering algorithm. Use a small dataset and apply different linkage methods such as single, complete, and average linkage. Generate a dendrogram and analyze how cluster formation changes with different linkage criteria.

Answer:

```
import numpy as np
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
import matplotlib.pyplot as plt

np.random.seed(2)
X = np.vstack([np.random.randn(8,2)+[1,1], np.random.randn(8,2)+[6,6], np.random.randn(8,2)+[1,6]])

for method in ['single', 'complete', 'average']:
    Z = linkage(X, method=method)
    clusters = fcluster(Z, t=3, criterion='maxclust')
    print(f"Linkage={method}: cluster assignment counts -> "
```

```
        f"{np.bincount(clusters)[1:]}")

    plt.figure()
    dendrogram(Z)
    plt.title(f"Dendrogram ({method} linkage)")
    plt.savefig(f"dendrogram_{method}.png")
```

Hierarchical clustering builds a tree (dendrogram) by repeatedly merging the two closest clusters, where 'closest' is defined differently by each linkage method. Single linkage merges clusters based on their nearest pair of points, which can produce elongated, chained clusters and is sensitive to noise ('chaining effect'). Complete linkage merges based on the farthest pair of points, tending to produce compact, evenly-sized, more balanced clusters. Average linkage uses the mean pairwise distance between clusters and produces a compromise between the two, usually giving stable, moderately compact clusters. Cutting the dendrogram at a chosen height (or number of clusters) yields the final partition; the height at which merges happen also visually indicates how distinct the natural clusters are in the data.

Q14. Create a Python program to implement a simple Artificial Neural Network (ANN) with one hidden layer. Train the network using forward propagation and backpropagation. Test the model on sample inputs and display predicted outputs. Analyze how the number of neurons and learning rate affect performance.

Answer:

```
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]])   # XOR problem
y = np.array([[0],[1],[1],[0]])

def sigmoid(z): return 1/(1+np.exp(-z))
def sigmoid_deriv(a): return a*(1-a)

def train_ann(hidden_neurons=4, lr=0.5, epochs=5000, seed=1):
    rng = np.random.RandomState(seed)
    W1 = rng.uniform(-1,1,(2, hidden_neurons))
    b1 = np.zeros((1, hidden_neurons))
    W2 = rng.uniform(-1,1,(hidden_neurons, 1))
    b2 = np.zeros((1,1))

    for epoch in range(epochs):
        # Forward propagation
        z1 = X @ W1 + b1
        a1 = sigmoid(z1)
        z2 = a1 @ W2 + b2
        a2 = sigmoid(z2)

        # Backpropagation
        error = y - a2
        d2 = error * sigmoid_deriv(a2)
        d1 = (d2 @ W2.T) * sigmoid_deriv(a1)
```

```
        W2 += lr * (a1.T @ d2)
        b2 += lr * d2.sum(axis=0, keepdims=True)
        W1 += lr * (X.T @ d1)
        b1 += lr * d1.sum(axis=0, keepdims=True)

    return a2

for hidden in [2, 4, 8]:
    for lr in [0.1, 0.5, 1.0]:
        output = train_ann(hidden_neurons=hidden, lr=lr)
        mse = np.mean((y - output)**2)
        print(f"hidden={hidden}, lr={lr} -> predictions={np.round(output.ravel(),2)} MSE={mse:.4f}")
```

The network passes inputs through a hidden layer of sigmoid units (forward propagation) and then propagates the output error backward, updating weights proportional to the learning rate and the local gradient (backpropagation). Because XOR is not linearly separable, a single-layer perceptron cannot solve it, but adding a hidden layer with enough neurons lets the network learn a non-linear decision surface. Too few hidden neurons (e.g. 2) can under-represent the required non-linearity and converge to a high error, while more neurons (4-8) give the network enough capacity to fit XOR closely. The learning rate again trades off speed and stability: very small rates converge slowly within the fixed epoch budget, and very large rates can cause oscillation, so a moderate rate (around 0.5) typically gives the best convergence for this small network.

Q15. Write a Python program to implement the Apriori algorithm for association rule mining. Use a transaction dataset to generate frequent itemsets and association rules. Display support, confidence, and lift values, and analyze how minimum support and confidence thresholds affect the results.

Answer:

```
# pip install mlxtend
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

transactions = [
    ['Bread','Milk'],
    ['Bread','Diaper','Beer','Eggs'],
    ['Milk','Diaper','Beer','Cola'],
    ['Bread','Milk','Diaper','Beer'],
    ['Bread','Milk','Diaper','Cola'],
]

te = TransactionEncoder()
te_data = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_data, columns=te.columns_)

for min_support in [0.3, 0.5, 0.6]:
    frequent_itemsets = apriori(df, min_support=min_support, use_colnames=True)
    print(f"\nmin_support={min_support}: {len(frequent_itemsets)} frequent itemsets found")
```

```
if not frequent_itemsets.empty:
    rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.6)
    print(rules[['antecedents','consequents','support','confidence','lift']])
```

The Apriori algorithm exploits the property that any subset of a frequent itemset must itself be frequent: it iteratively builds larger candidate itemsets from smaller frequent ones and prunes candidates whose support falls below the minimum support threshold, then generates association rules from the surviving itemsets, evaluated by confidence and lift. Raising the minimum support threshold sharply reduces the number of frequent itemsets found (since only very common combinations survive), which speeds up computation but may miss rare-but-important associations; lowering it reveals more itemsets and rules but increases computation and the chance of spurious, low-value rules. Similarly, raising the minimum confidence threshold keeps only the strongest, most reliable rules (high probability that the consequent occurs given the antecedent), while a low threshold retains weaker rules. Lift values above 1 indicate the items co-occur more than would be expected if they were independent, confirming a genuinely useful association rather than a coincidental one.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand